Báo Cáo Bài Tập Lớn: Kỹ thuật lập trình

Đề tài: Quản lí thiết bị di động

Phần thực hiện: Thiết bị đầu cuối IMEI và các thư viện dùng chung/liên quan

Sinh viên thực hiện: Trần Đức Anh Mã sinh viên: B24DCVT021 Lớp: D24CQVT01-B

Chương 1. Giới thiệu và phân tích yêu cầu

1.1 Mô tả bài toán

Trong các hệ thống viễn thông của các nhà mạng luôn có các bộ phận quản lí khách hàng, thuê bao, hợp đồng, SIM và thiết bị di động. Khi khách hàng đăng kí dịch vụ, đổi thiết bị, thay SIM, khóa máy bị mất hoặc cần tra cứu thông tin, nhà mạng cần có hệ thống lưu trữ và xử lí dữ liệu chính xác.

Trong bài tập lớn này, chương trình được xây dựng theo hướng mô phỏng một hệ thống quản lí thuê bao di động. Phần em thực hiện tập trung vào quản lí hợp đồng dịch vụ và quản lí thiết bị đầu cuối IMEI. Ngoài ra, em xây dựng các thư viện dùng chung để các thành viên khác có thể sử dụng khi tích hợp các phân hệ của mình.

Thiết bị đầu cuối được nhận biết bằng mã IMEI. Đây là một chuỗi gồm 15 chữ số. Chương trình kiểm tra mã IMEI bằng thuật toán Luhn Mod-10 trước khi cho phép lưu vào hệ thống. Mỗi thiết bị có thể được gán với một số điện thoại, được cập nhật trạm BTS gần nhất và được chuyển sang trạng thái khóa mạng khi cần thiết.

Chương trình hiện tại là chương trình console chạy cục bộ phục vụ mục đích học tập. Chương trình chưa kết nối với hệ thống mạng di động thật, EIR thật hoặc cơ sở dữ liệu của nhà mạng.

1.2 Các chức năng thực hiện trong chương trình

Phần thực hiện gồm hai nhóm chức năng chính:

a) Quản lí hợp đồng dịch vụ

Thêm hợp đồng mới.

Xem danh sách hợp đồng.

Tìm kiếm hợp đồng theo mã hợp đồng, số điện thoại hoặc mã khách hàng.

Sắp xếp hợp đồng theo ngày đăng kí hoặc giá trị gói cước.

Cập nhật, gia hạn hợp đồng.

Tạm dừng, kích hoạt lại hoặc chấm dứt hợp đồng.

Xóa hợp đồng sau khi xác nhận.

b) Quản lí thiết bị đầu cuối IMEI

Thêm thiết bị mới.

Kiểm tra mã IMEI có đủ 15 chữ số và đúng Luhn hay không.

Xem danh sách thiết bị.

Tìm kiếm thiết bị theo mã IMEI, số điện thoại hoặc hãng sản xuất. Mã BTS chỉ được lưu như trạm gần nhất; chương trình chưa đo chất lượng 5G hoặc chất lượng mạng.

Gán SIM cho thiết bị.

Gỡ SIM khỏi thiết bị.

Cập nhật mã trạm BTS gần nhất.

Khóa hoặc mở khóa thiết bị.

Xem danh sách thiết bị đang bị khóa mạng.

Xóa thiết bị khỏi hệ thống.

1.3 Yêu cầu đối với dữ liệu IMEI

Mã IMEI được kiểm tra theo các điều kiện sau:

Có đúng 15 kí tự.

Tất cả các kí tự đều là chữ số từ 0 đến 9.

Không được là chuỗi toàn số 0.

Tổng kiểm tra phải thỏa mãn thuật toán Luhn Mod-10.

Không được trùng với mã IMEI đã tồn tại trong kho dữ liệu.

Trong cấu trúc IMEI hiện đại, 15 chữ số thường được chia thành TAC, SNR và chữ số kiểm tra. Trong chương trình, toàn bộ 15 chữ số được lưu thành một mã định danh duy nhất để thuận tiện cho việc tìm kiếm và lưu trữ.

1.4 Yêu cầu đối với số điện thoại và trạng thái thiết bị

Số điện thoại di động phải có 10 chữ số và bắt đầu bằng một trong các đầu số 03, 05, 07, 08 hoặc 09. Khi thiết bị chưa được gán SIM, chương trình sử dụng giá trị ChuaGan.

Các trạng thái thiết bị được sử dụng trong chương trình gồm:

HoatDong: thiết bị đang hoạt động.

KhoaMang: thiết bị bị khóa mạng hoặc bị đưa vào danh sách đen mô phỏng.

TamKhoa: thiết bị tạm khóa.

Ngày kích hoạt thiết bị và các ngày trong hợp đồng được kiểm tra bằng thư viện Date. Ngày không hợp lệ, ngày hết hạn nhỏ hơn ngày đăng kí hoặc ngày sai định dạng đều bị từ chối.

1.5 Yêu cầu chung của hệ thống

Dữ liệu nhập vào phải được kiểm tra trước khi lưu.

Không cho phép mã bị trùng.

Không để chương trình bị dừng khi người dùng nhập sai kiểu dữ liệu.

Dữ liệu sau khi thêm, sửa hoặc xóa phải được lưu lại trong file.

Các lớp nghiệp vụ không phụ thuộc trực tiếp vào cách nhập dữ liệu từ bàn phím.

Mã nguồn được chia thành các file riêng để dễ đọc, kiểm thử và bảo trì.

Chương 2. Môi trường và công cụ lập trình

2.1 Môi trường và công cụ lập trình

Hệ điều hành: Linux, phân phối Debian 13.

Ngôn ngữ lập trình: C++.

Trình biên dịch: GCC/G++. Makefile của dự án sử dụng chuẩn C++11 với các cờ cảnh báo -Wall và -Wextra. GCC trên máy có thể hỗ trợ các chuẩn C++ mới hơn, nhưng mã nguồn hiện tại được biên dịch theo cấu hình của Makefile.

Môi trường soạn thảo: Visual Studio Code và Neovim. Trong quá trình thực hiện, Visual Studio Code được sử dụng nhiều hơn vì có terminal, extension hỗ trợ C++ và khả năng quản lí nhiều file trong dự án.

Công cụ biên dịch và chạy chương trình: GNU Make và terminal.

Công cụ phối hợp với các thành viên khác: Git và GitHub. Mỗi thành viên có một nhánh riêng dựa theo tên và mã sinh viên. Em có nhiệm vụ đọc lại code, chạy kiểm thử, kiểm tra lỗi và tích hợp các thay đổi khi phù hợp.

Công cụ kiểm thử: File tests/test_runner.cpp chứa các kiểm tra assertion cho Date, số điện thoại, Normalized, IMEI, hợp đồng, DataStore, DisplayHelper và các menu.

Các lệnh chính để biên dịch, chạy và kiểm thử chương trình:

make all

make test

make run

make clean

2.2 Danh sách thư viện chuẩn đã dùng và mục tiêu

a) Thư viện vào, nhập xuất và định dạng

Dùng để nhập dữ liệu từ bàn phím và hiển thị kết quả ra màn hình console thông qua std::cin và std::cout.

Dùng để tạo chuỗi file và xử lí dữ liệu dạng chuỗi trong quá trình chuyển đổi đối tượng thành bản ghi.

Dùng để căn chỉnh độ rộng các cột khi hiển thị danh sách hợp đồng và thiết bị.

Dùng để đọc và ghi dữ liệu từ các file văn bản.

Dùng cho các thao tác file như đổi tên file tạm thành file dữ liệu chính.

b) Loại dữ liệu và bộ nhớ

Dùng để lưu mã IMEI, tên thiết bị, hãng sản xuất, số điện thoại, trạng thái và các mã nghiệp vụ.

Dùng để lưu danh sách các đối tượng trong bộ nhớ của DataStore.

Dùng để biểu diễn các cặp dữ liệu, ví dụ nhãn và giá trị khi in thẻ chi tiết.

c) Tiện ích và thuật toán

Dùng cho các thao tác sắp xếp, tìm kiếm và xử lí danh sách.

Dùng để truyền điều kiện lọc và hàm so sánh cho DataStore.

Dùng để kiểm tra và xử lí các kí tự trong quá trình chuẩn hóa dữ liệu.

d) Nhận biết ngày tháng và xử lí lỗi

Dùng để lấy ngày hiện tại và hỗ trợ các thao tác liên quan đến ngày tháng.

Dùng làm lớp cơ sở cho các ngoại lệ trong hệ thống như lỗi IMEI, lỗi số điện thoại, lỗi ngày và lỗi file.

2.3 Các thư viện tự tạo và các hàm bên trong

Các thư viện tự tạo được chia thành ba tầng chính: Shared, Models và Menus.

a) Tầng thứ nhất: Shared

Đây là các thư viện dùng chung trong toàn bộ bài. Các thư viện ở tầng này cung cấp chức năng nền tảng cho các lớp ở tầng trên và không phụ thuộc vào menu nghiệp vụ.

Entity.h

Đây là lớp cơ sở dùng chung cho các thực thể. Lớp này chứa mã định danh và quy định các hàm hiển thị, đọc file và ghi file mà các lớp con phải cài đặt.

Exceptions.h

Chứa các loại ngoại lệ của chương trình:

AppException.

DuplicateIdException.

NotFoundException.

InvalidDateException.

InvalidLuhnException.

InvalidPhoneNumberException.

FileIOException.

Date.h/Date.cpp

Chức năng:

Kiểm tra ngày hợp lệ.

Kiểm tra năm nhuận.

Tính số ngày trong tháng.

Phân tích ngày từ chuỗi.

Chuyển ngày thành chuỗi dạng DD/MM/YYYY.

Tính tuổi.

So sánh ngày.

Kiểm tra ngày tương lai hoặc ngày quá khứ.

fileio.h/fileio.cpp

Chức năng:

Tách một dòng dữ liệu theo kí tự phân cách.

Xử lí khoảng trắng khi đọc token.

Tạo thư mục dữ liệu nếu chưa tồn tại.

Khai báo đường dẫn file và các hằng số liên quan đến file.

Normalized.h/Normalized.cpp

Chức năng: chuẩn hóa và kiểm tra dữ liệu đầu vào.

Các hàm chính:

trim: Xóa khoảng trắng ở đầu và cuối chuỗi.

CollapseSpace: Gộp nhiều dấu cách liên tiếp thành một dấu cách.

removeSymbols: Xóa hoặc xử lí các kí tự đặc biệt không được phép.

removeDigits: Xóa các chữ số trong những trường chỉ cho phép dữ liệu dạng tên.

toLower và toUpper: Chuyển đổi chữ hoa, chữ thường.

NormalizedName: Chuẩn hóa tên để hiển thị.

isValidPhoneNumber: Kiểm tra số điện thoại di động Việt Nam 10 chữ số.

InputHelper.h/InputHelper.cpp

Chức năng: nhập dữ liệu an toàn từ console.

Các hàm chính:

getString: Nhập chuỗi.

getInt: Nhập số nguyên trong khoảng cho phép.

getDouble: Nhập số thực trong khoảng cho phép.

getDate: Nhập ngày hợp lệ.

getBirthDate: Nhập ngày sinh theo giới hạn tuổi.

getPhoneNumber: Nhập và kiểm tra số điện thoại.

getConfirm: Nhập lựa chọn xác nhận.

pause: Tạm dừng màn hình.

DisplayHelper.h

Chức năng: hiển thị giao diện console thống nhất.

Các hàm chính:

printBorder: In đường viền bảng.

printHeader: In tiêu đề các cột.

printRow: In một dòng dữ liệu.

printCard: In thông tin dạng thẻ chi tiết.

DataStore.h

Chức năng: quản lí danh sách đối tượng trong bộ nhớ và lưu trữ xuống file.

Các hàm chính:

loadFromFile.

saveToFile.

add.

update.

remove.

findById.

filter.

sort.

clear.

BTSRegister.h/BTSRegister.cpp

Chức năng dự kiến: mô tả cấu trúc và giao diện quản lí danh sách trạm BTS. Phần cài đặt hiện tại còn là placeholder, chưa được nối vào Makefile hoặc luồng xử lí chính. Thiết bị IMEI hiện chỉ lưu mã trạm BTS gần nhất thay vì thực hiện kết nối vô tuyến thật.

b) Tầng thứ hai: Models

Tầng Models chứa các đối tượng nghiệp vụ chính:

hopdong.h/hopdong.cpp

Quản lí hợp đồng đăng kí dịch vụ, ngày đăng kí, ngày hết hạn, giá trị gói, loại hợp đồng và trạng thái hợp đồng.

imei.h/imei.cpp

Quản lí thiết bị đầu cuối IMEI. Lớp ThietBiIMEI kế thừa Entity và chứa các thông tin về mã IMEI, tên thiết bị, hãng sản xuất, số điện thoại, ngày kích hoạt, trạng thái và trạm BTS gần nhất.

c) Tầng thứ ba: Menus

Tầng Menus điều khiển giao diện console và gọi các chức năng của Models, InputHelper và DataStore.

HopDongMenu.h/HopDongMenu.cpp

Điều khiển các chức năng thêm, xem, tìm kiếm, sắp xếp, cập nhật và xóa hợp đồng.

ThietBiIMEIMenu.h/ThietBiIMEIMenu.cpp

Điều khiển các chức năng thêm, xem, tìm kiếm, khóa mạng, gán SIM, gỡ SIM, cập nhật BTS và xóa thiết bị.

Chương 3. Thiết kế giải pháp

3.1 Giải pháp tổng thể

Hệ thống được thiết kế theo hướng lập trình hướng đối tượng và chia thành các tầng để tránh trộn lẫn giao diện với xử lí nghiệp vụ.

Tầng Menu tiếp nhận lựa chọn của người dùng. Tầng Models kiểm tra và xử lí các quy tắc nghiệp vụ. Tầng Shared cung cấp các hàm dùng chung như chuẩn hóa chuỗi, kiểm tra ngày, kiểm tra số điện thoại, hiển thị và xử lí ngoại lệ. Tầng DataStore và FileIO chịu trách nhiệm lưu trữ dữ liệu.

Sơ đồ phụ thuộc chính:

Menu ↓ Models ↓ Shared validation và Date ↓ DataStore và FileIO

Cách tổ chức này giúp lớp ThietBiIMEI có thể được kiểm thử trực tiếp mà không cần nhập dữ liệu từ bàn phím. Khi cần phát triển giao diện khác, phần model có thể được sử dụng lại.

3.2 Thiết kế xử lí IMEI

Hàm ThietBiIMEI::validateLuhn kiểm tra IMEI theo các bước:

Kiểm tra chuỗi có đúng 15 kí tự hay không.

Kiểm tra từng kí tự có phải chữ số hay không.

Từ chối chuỗi toàn số 0.

Duyệt từ phải sang trái.

Nhân đôi mỗi chữ số thứ hai tính từ bên phải.

Nếu kết quả lớn hơn 9 thì trừ 9.

Kiểm tra tổng cuối cùng có chia hết cho 10 hay không.

Hàm được gọi ở constructor và setMaIMEI. Nhờ vậy, mã IMEI sai không thể được đưa vào đối tượng thông qua lúc thêm mới hoặc lúc cập nhật.

3.3 Thiết kế gán SIM và quản lí trạng thái

Hàm ganSIM kiểm tra số điện thoại trước khi gán vào thiết bị. Nếu số điện thoại không hợp lệ, chương trình phát sinh InvalidPhoneNumberException.

Hàm goSIM gỡ số điện thoại và đưa giá trị về ChuaGan.

Hàm setBlacklist nhận giá trị đúng hoặc sai. Nếu đúng, trạng thái thiết bị chuyển thành KhoaMang. Nếu sai, trạng thái chuyển về HoatDong.

Hàm capNhatBTS cập nhật mã trạm BTS gần nhất cho thiết bị.

Trong hệ thống nhà mạng thật, EIR có thể có danh sách trắng, danh sách xám và danh sách đen. Trong bài tập này, KhoaMang chỉ là trạng thái mô phỏng ở mức chương trình và không gửi lệnh đến EIR thật.

3.4 Lưu trữ dữ liệu bằng file

Chương trình sử dụng file văn bản trong thư mục data. Mỗi dòng là một bản ghi và các trường được phân cách bởi kí tự |.

Bản ghi thiết bị IMEI có dạng:

IMEI|Tên thiết bị|Hãng sản xuất|Số điện thoại|Ngày kích hoạt|Trạng thái|Trạm BTS gần nhất

DataStore đọc dữ liệu vào vector trong bộ nhớ. Khi đọc file, chương trình bỏ qua dòng trống và dòng bắt đầu bằng dấu #. Sau khi thêm, sửa hoặc xóa, chương trình ghi dữ liệu vào file tạm có phần mở rộng .tmp rồi đổi tên thành file chính.

Giải pháp dùng file được lựa chọn vì bài tập có quy mô nhỏ, dễ triển khai, dễ kiểm tra bằng mắt và không cần cài đặt hệ quản trị cơ sở dữ liệu. Nhược điểm là toàn bộ dữ liệu được đọc vào bộ nhớ, thao tác ghi lại cả file và chưa hỗ trợ nhiều người dùng cùng lúc.

3.5 Một số rủi ro và hướng giải quyết

a) File dữ liệu chưa tồn tại

Ở lần chạy đầu tiên, thư mục dữ liệu được tạo nếu cần nhưng file dữ liệu phải tồn tại và mở được trước khi chương trình tiếp tục. Nếu không thể mở một file dữ liệu, loadFromFile trả về false; main báo lỗi và dừng chương trình với mã khác 0 để tránh chạy trên trạng thái dữ liệu không xác định.

b) Dữ liệu trong file sai định dạng

Một dòng có thể bị thiếu trường, có thêm trường, sai IMEI, sai ngày, sai trạng thái, sai giá trị gói hoặc sai số điện thoại. Hàm fromFileString kiểm tra đúng số lượng trường, dựng lại đối tượng qua constructor có kiểm tra nghiệp vụ và bắt lỗi khi phân tích dữ liệu. Dòng không hợp lệ không được đưa vào danh sách sử dụng.

c) Nhập IMEI sai

IMEI sai độ dài, chứa chữ, toàn số 0 hoặc sai checksum sẽ bị từ chối bằng InvalidLuhnException.

d) Nhập số điện thoại sai

Số điện thoại sai độ dài hoặc sai đầu số sẽ bị từ chối bằng InvalidPhoneNumberException.

e) Trùng mã dữ liệu

DataStore kiểm tra mã trước khi thêm. Nếu mã IMEI hoặc mã hợp đồng đã tồn tại, chương trình phát sinh DuplicateIdException.

f) Cập nhật hoặc xóa bản ghi không tồn tại

Nếu không tìm thấy mã cần sửa hoặc xóa, chương trình phát sinh NotFoundException và hiển thị thông báo cho người dùng.

g) Lỗi trong quá trình ghi file

Nếu file tạm không thể ghi hoặc không thể đổi tên, chương trình phát sinh FileIOException. Việc ghi file tạm giúp giảm nguy cơ làm hỏng file dữ liệu chính, nhưng đây vẫn chưa phải cơ chế giao dịch đầy đủ như trong cơ sở dữ liệu.

h) Xóa dữ liệu đang được tham chiếu

Khi các phân hệ khác được tích hợp đầy đủ, cần kiểm tra quan hệ giữa thuê bao, hợp đồng, SIM, thiết bị và gói cước trước khi xóa. Nếu còn dữ liệu phụ thuộc, có thể từ chối xóa hoặc chuyển đối tượng sang trạng thái ngừng hoạt động để giữ lịch sử.

3.6 Kiểm thử chương trình

Bộ kiểm thử nằm trong tests/test_runner.cpp. Các nhóm kiểm thử gồm:

Kiểm tra Date và năm nhuận.

Kiểm tra ngày sinh và giới hạn tuổi.

Kiểm tra số điện thoại di động Việt Nam.

Kiểm tra các hàm Normalized.

Kiểm tra IMEI và thuật toán Luhn.

Kiểm tra hợp đồng và thời hạn hợp đồng.

Kiểm tra CRUD và lưu file của DataStore.

Kiểm tra DisplayHelper.

Kiểm tra các menu hợp đồng và thiết bị IMEI.

Kiểm tra nạp file có bản ghi sai và file dữ liệu không tồn tại.

Lệnh chạy kiểm thử:

make test

Kết quả hiện tại:

Kết quả kiểm thử: 144 passed, 0 failed (số lượng có thể thay đổi khi bổ sung kiểm thử mới).

Tất cả kiểm thử đã vượt qua thành công, đạt 100% trong phạm vi bộ test hiện có.

Lệnh make all đã biên dịch chương trình thành công sau các thay đổi. Kiểm tra biên dịch nghiêm ngặt với -std=c++11 -pedantic-errors cũng đã hoàn thành.

3.7 Cơ hội phát triển hệ thống

Bổ sung đầy đủ các module quản lí thuê bao, gói cước, nạp tiền và hóa đơn.

Quản lí nhiều nhà mạng và danh sách đầu số bằng dữ liệu cấu hình.

Tách TAC để tra cứu hãng và model thiết bị.

Bổ sung white list, grey list và lịch sử khóa/mở khóa thiết bị.

Xuất báo cáo thống kê số thiết bị theo hãng, trạng thái hoặc trạm BTS.

Thay file văn bản bằng SQLite hoặc cơ sở dữ liệu quan hệ khi dữ liệu tăng lên.

Hoàn thiện BTSRegister và bổ sung chức năng tra cứu thiết bị theo mã trạm BTS nếu yêu cầu nghiệp vụ cần đến.

Thêm tài khoản, phân quyền, nhật kí thao tác và mã hóa dữ liệu cá nhân.

Phát triển giao diện đồ họa hoặc API dựa trên các lớp nghiệp vụ hiện tại.

Chương 4. Kiến thức mới tự tìm hiểu và vận dụng

4.1 Kiến thức mới: Xử lí ngoại lệ trong C++

a) Học gì

Em tìm hiểu cơ chế try, throw và catch trong C++. Ngoài các ngoại lệ chuẩn, chương trình có thể tạo các lớp ngoại lệ riêng để mô tả từng loại lỗi trong nghiệp vụ.

b) Nguồn ở đâu

Nguồn tìm hiểu gồm tiêu chuẩn C++, tài liệu về std::runtime_error, tài liệu GCC và mã nguồn Exceptions.h của dự án.

c) Áp dụng vào đâu

Các lỗi InvalidLuhnException, InvalidPhoneNumberException, InvalidDateException, DuplicateIdException, NotFoundException và

FileIOException được dùng ở các lớp nghiệp vụ và DataStore. Menu bắt lỗi và thông báo cho người dùng thay vì để chương trình tiếp tục với dữ liệu sai.

d) Lợi ích

Xử lí ngoại lệ giúp chương trình ổn định hơn, phân biệt được nhiều loại lỗi và dễ mở rộng khi thêm nghiệp vụ mới.

4.2 Kiến thức mới: Cấu trúc IMEI và thuật toán Luhn

a) Học gì

Em tìm hiểu IMEI là mã nhận dạng thiết bị di động gồm 15 chữ số. Cấu trúc hiện đại thường có TAC, SNR và chữ số kiểm tra. Chữ số kiểm tra được xác nhận bằng thuật toán Luhn Mod-10.

b) Nguồn ở đâu

Nguồn tìm hiểu gồm 3GPP TS 23.003, 3GPP TS 22.016 và tài liệu GSMA TS.06 về phân bổ IMEI.

c) Áp dụng vào đâu

Hàm ThietBiIMEI::validateLuhn được gọi khi tạo mới và cập nhật mã IMEI. Bộ test kiểm tra IMEI hợp lệ, IMEI ngắn, IMEI dài, IMEI chứa chữ, IMEI toàn số 0 và IMEI sai checksum.

d) Lợi ích

Thuật toán giúp không lưu mã IMEI sai và giúp bài toán gần với nghiệp vụ quản lí thiết bị viễn thông thực tế hơn.

4.3 Kiến thức mới: Tách xử lí nhập liệu và xử lí nghiệp vụ

a) Học gì

Em tìm hiểu cách tách phần nhập dữ liệu từ console khỏi phần kiểm tra nghiệp vụ. InputHelper phụ trách đọc và yêu cầu nhập lại. Normalized và các Model phụ trách xác định dữ liệu có hợp lệ hay không.

b) Nguồn ở đâu

Nguồn tìm hiểu gồm tài liệu lập trình hướng đối tượng C++, tài liệu nội bộ trong docs/Overview/lib.md và đặc tả luồng gọi hàm của dự án.

c) Áp dụng vào đâu

InputHelper gọi Normalized::isValidPhoneNumber và các model cũng tự kiểm tra số điện thoại khi constructor hoặc mutator được gọi. Khi đọc file, fromFileString dựng lại đối tượng qua constructor nên dữ liệu từ menu, file hoặc test đều đi qua quy tắc nghiệp vụ.

d) Lợi ích

Cách tách này làm code dễ kiểm thử, giảm code lặp và cho phép sau này thay giao diện console bằng giao diện khác mà không phải viết lại Model.

4.4 Kiến thức mới: Lưu file tạm và kiểm thử hồi quy

a) Học gì

Em tìm hiểu rủi ro khi ghi trực tiếp lên file chính và cách ghi ra file tạm trước khi đổi tên. Em cũng tìm hiểu vai trò của kiểm thử hồi quy sau khi sửa thư viện dùng chung.

b) Nguồn ở đâu

Nguồn tìm hiểu gồm tài liệu C++ về ifstream, ofstream, rename, tài liệu GNU Make và mã nguồn DataStore.h, fileio.h, tests/test_runner.cpp.

c) Áp dụng vào đâu

DataStore::saveToFile ghi dữ liệu vào file .tmp trước khi thay file chính. Bộ test kiểm tra thêm, sửa, xóa, mã trùng, mã không tồn tại, lưu file và nạp lại dữ liệu.

d) Lợi ích

Giải pháp giúp giảm nguy cơ làm hỏng file dữ liệu khi chương trình bị gián đoạn và giúp phát hiện lỗi sau mỗi lần thay đổi mã nguồn.

KẾT LUẬN

Bài tập lớn đã xây dựng được phần chương trình C++ mô phỏng quản lí hợp đồng dịch vụ và thiết bị đầu cuối trong hệ thống thuê bao di động. Phần thực hiện của em tập trung vào thiết bị IMEI, kiểm tra Luhn Mod-10, gán và gỡ SIM, khóa thiết bị, cập nhật trạm BTS và các thư viện dùng chung.

Việc chia chương trình thành các tầng Shared, Models và Menus giúp mã nguồn rõ ràng hơn. Các lớp nghiệp vụ có thể được kiểm thử độc lập, dữ liệu được kiểm tra trước khi lưu và các lỗi được biểu diễn bằng ngoại lệ phù hợp.

Kết quả chạy bộ kiểm thử hiện tại là 144 passed, 0 failed (số lượng có thể thay đổi khi bổ sung kiểm thử mới). Chương trình phù hợp với phạm vi bài tập và

có thể tiếp tục mở rộng bằng cách tích hợp các module thuê bao, gói cước, nạp tiền, hóa đơn, cơ sở dữ liệu và các chức năng bảo mật.

TÀI LIỆU THAM KHẢO

1. 3GPP TS 23.003: Numbering, addressing and identification. https://www.3gpp.org/ftp/Specs/archive/23_series/23.003/
2. 3GPP TS 22.016: International Mobile station Equipment Identities. https://www.3gpp.org/ftp/Specs/archive/22_series/22.016/
3. GSMA TS.06: IMEI Allocation and Approval Process. https://www.gsma.com/get-involved/working-groups/gsma_resources/ts-06
4. ISO/IEC 14882:2017 C++ language standard. https://www.iso.org/standard/68564.html
5. GNU Make manual. https://www.gnu.org/software/make/manual/make.html
6. Git user manual. https://git-scm.com/docs/user-manual
7. Mã nguồn và tài liệu nội bộ của dự án: README.md, Makefile, docs/Overview, docs/Operational, src/lib và tests/test_runner.cpp.